



ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

CLASE 3 - GRASP



¿DISEÑO DE OBJETOS? UN “CONSEJO”...

Después de la **identificación de los requisitos** y la creación de un **modelo del dominio**, entonces se añade **métodos** a las clases de software y se define el **paso de mensajes** entre los objetos para **satisfacer** los requisitos.

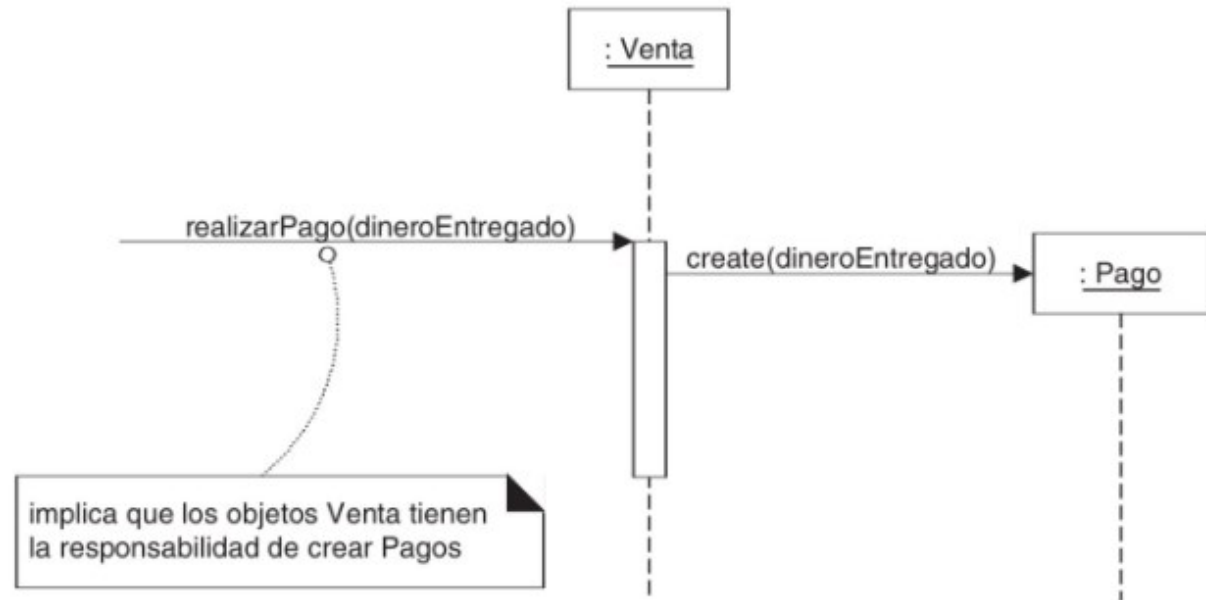
PATRONES GRASP (DE ASIGNACIÓN DE RESPONSABILIDADES)

- ¿Qué es una responsabilidad?
 - Según UML es un “contrato u obligación de un clasificador” (obligación de un objeto en cuanto a su **comportamiento**)
- Tipos de responsabilidades:
 - Conocer (datos privados encapsulados, objetos relacionados, cosas que puede calcular)
 - Hacer (el mismo, iniciar acción en otros objetos, controlar y coordinar actividades entre objetos)

La responsabilidad **NO** necesariamente es un método

DIAGRAMAS DE INTERACCIÓN

Reflejan las decisiones de asignación de responsabilidades



¿QUÉ ES UN PATRÓN?

- Descripción de problema y solución con nombre
- Esta descripción es aplicable en nuevos contextos
- Codifica conocimiento, estilos y principios que **ya existen** y se demostraron **válidos**

“Un patrón de una persona es un bloque de construcción primitivo de otra”

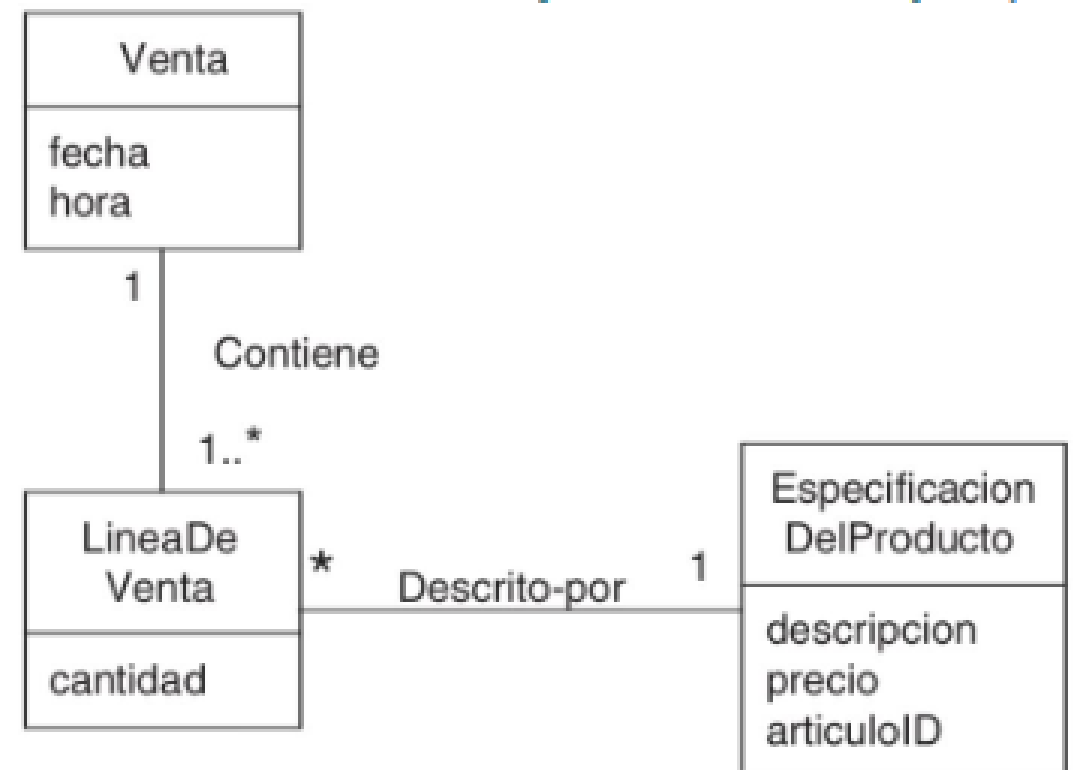
GRASP

- **Experto en información** (Information expert)
- **Creador** (Creator)
- **Bajo acoplamiento** (Low coupling)
- **Alta cohesión** (High cohesion)
- **Controlador** (Controller)

GRASP

EXPERTO EN INFORMACIÓN

Problema	Solución	Beneficios
¿Cómo asignamos responsabilidades a los objetos?	Asignarlas a la clase que tiene la información necesaria para la responsabilidad (experto)	Sistemas más fáciles de mantener; entender. Se mantiene información encapsulada

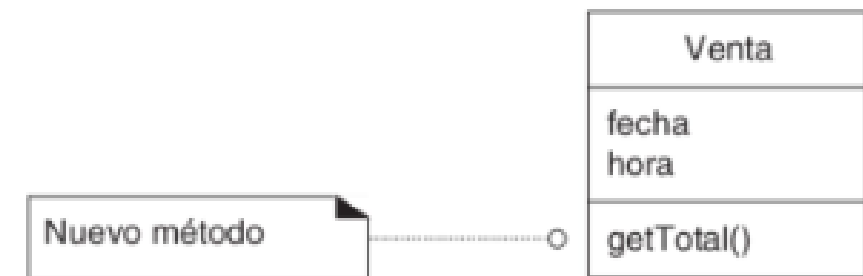
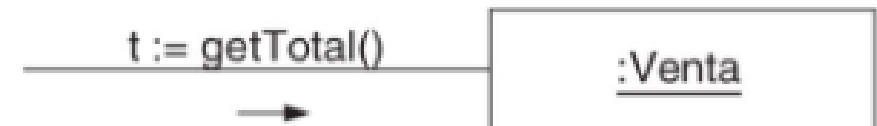
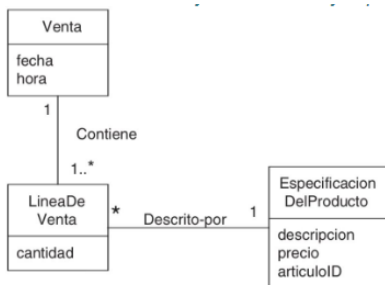


¿Quién debería conocer el total de una venta?

GRASP

EXPERTO EN INFORMACIÓN

Problema	Solución	Beneficios
¿Cómo asignamos responsabilidades a los objetos?	Asignarlas a la clase que tiene la información necesaria para la responsabilidad (experto)	Sistemas más fáciles de mantener, entender. Se mantiene información encapsulada

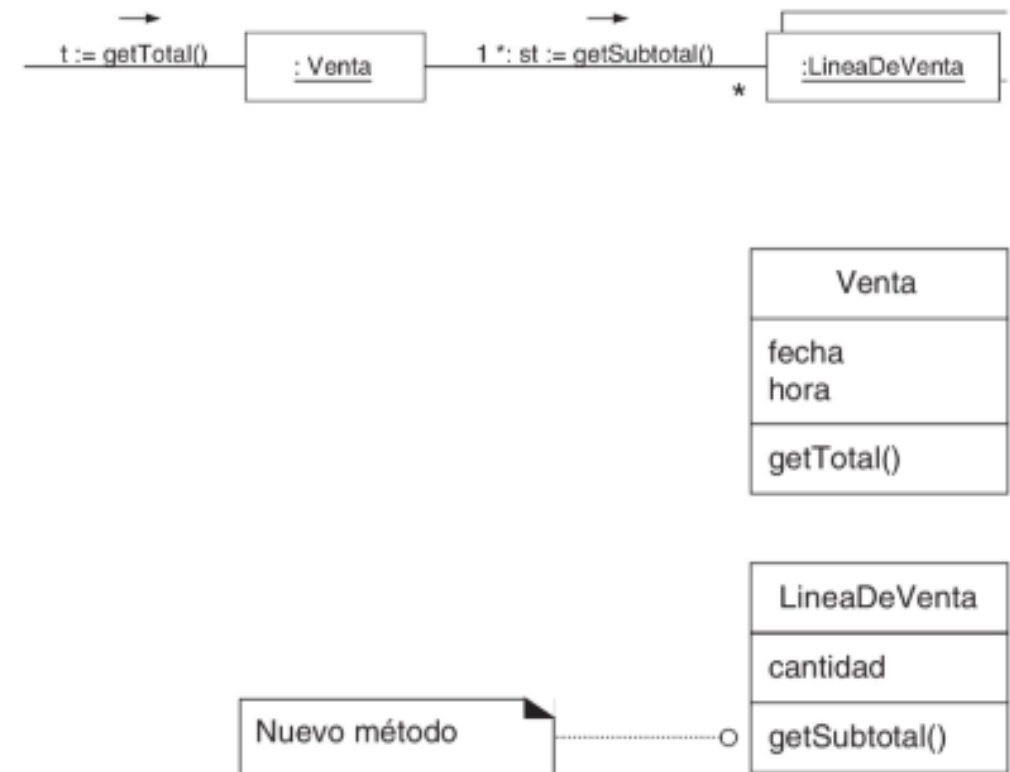
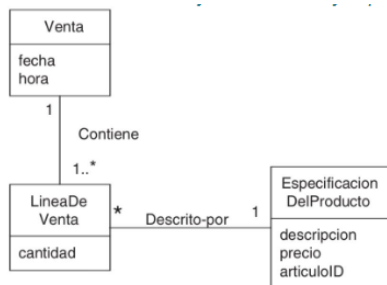


¿Quién debería conocer el total de una venta?

GRASP

EXPERTO EN INFORMACIÓN

Problema	Solución	Beneficios
¿Cómo asignamos responsabilidades a los objetos?	Asignarlas a la clase que tiene la información necesaria para la responsabilidad (experto)	Sistemas más fáciles de mantener, entender. Se mantiene información encapsulada

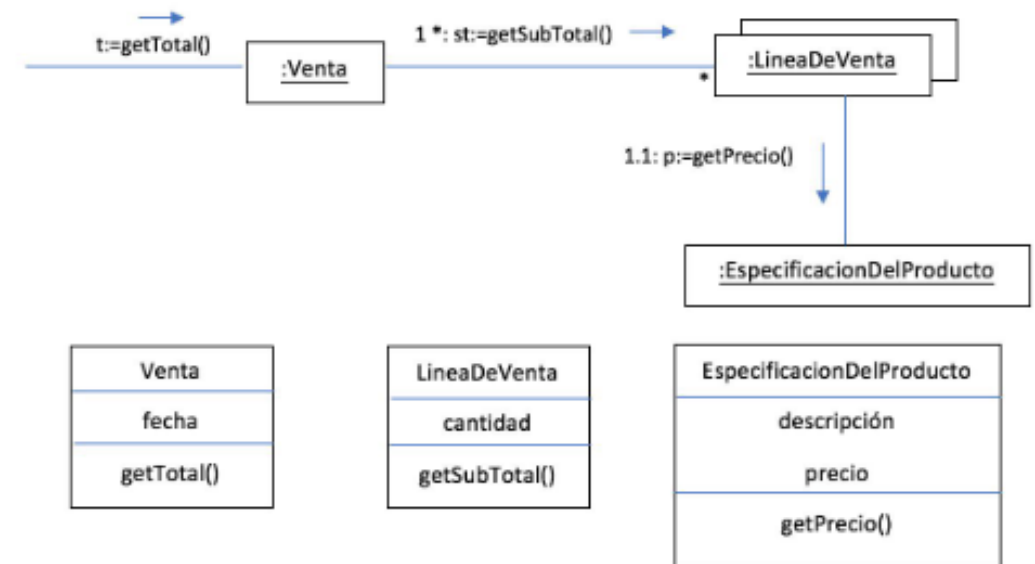
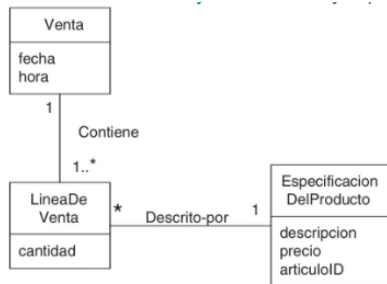


¿Quién debería conocer el total de una venta?

GRASP

EXPERTO EN INFORMACIÓN

Problema	Solución	Beneficios
¿Cómo asignamos responsabilidades a los objetos?	Asignarlas a la clase que tiene la información necesaria para la responsabilidad (experto)	Sistemas más fáciles de mantener, entender. Se mantiene información encapsulada



Clase

Responsabilidad

Venta

Conocer el total de la venta

LineaDeVenta

Conocer el subtotal de la línea de venta

EspecificacionDelProducto

Conocer el precio del producto

GRASP CREADOR

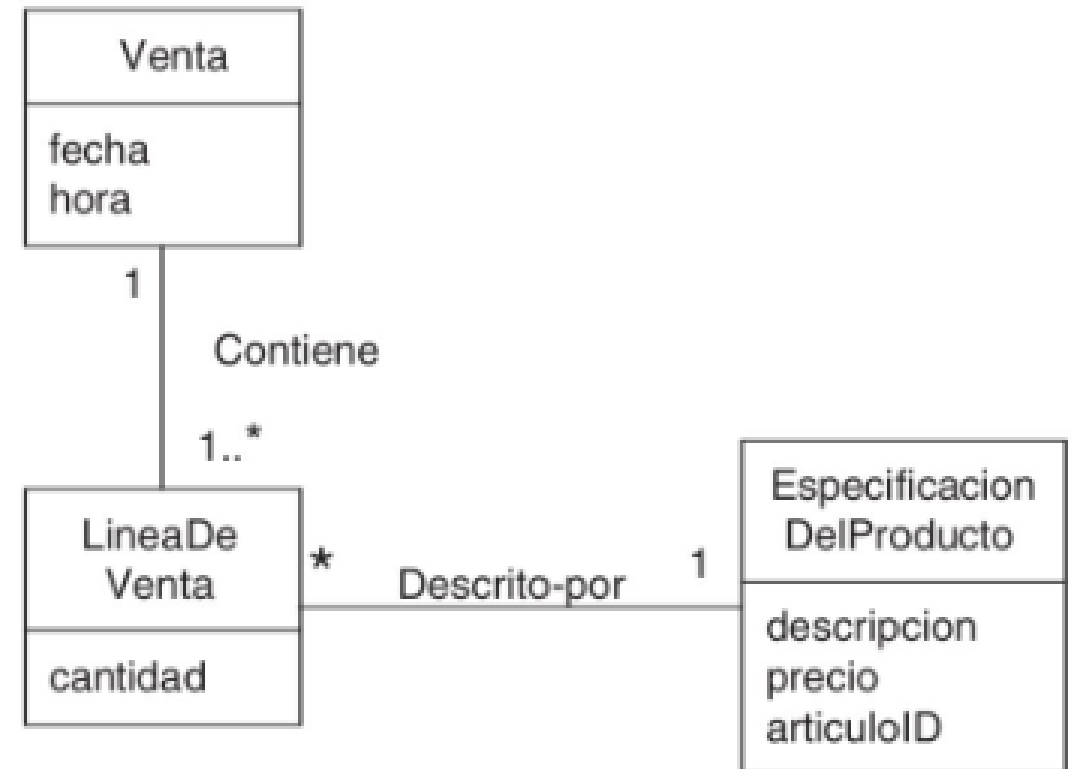
Problema

¿Quién es responsable de crear nuevas instancias de una clase?

Solución

La clase B es responsable de crear una clase A si:

- B **agrega** objetos de A
- B **contiene** objetos de A
- B **registra** instancias de objetos de A
- B **utiliza más estrechamente** objetos de A
- B **tiene datos de inicialización** que se pasaran a un objeto de A



¿Quién debería crear LineaDeVenta?

GRASP CREADOR

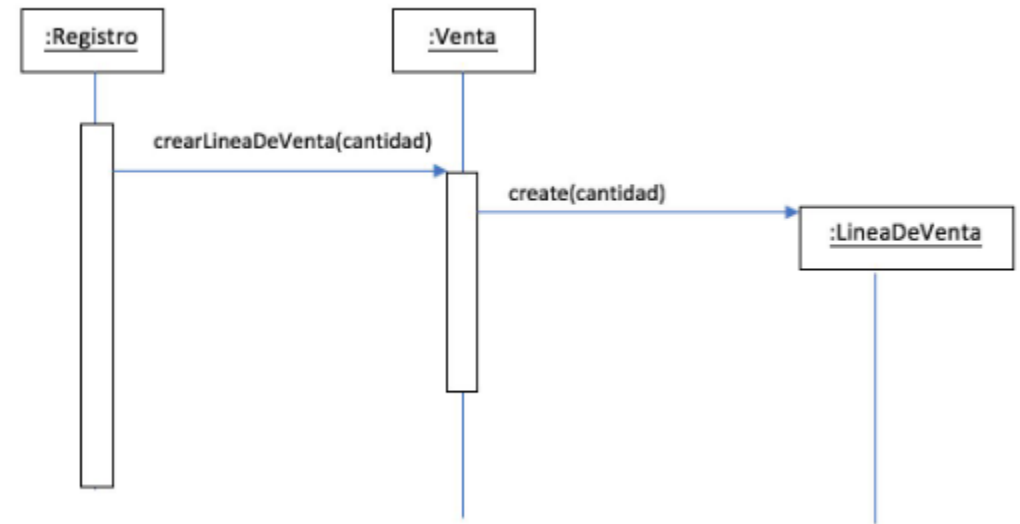
Problema

¿Quién es responsable de crear nuevas instancias de una clase?

Solución

La clase B es responsable de crear una clase A si:

- B **agrega** objetos de A
- B **contiene** objetos de A
- B **registra** instancias de objetos de A
- B **utiliza más estrechamente** objetos de A
- B **tiene datos de inicialización** que se pasaran a un objeto de A



¿Quién debería crear LineaDeVenta?

GRASP

BAJO ACOPLAMIENTO

Problema	Solución	Beneficios
¿Cómo soportar baja dependencias, bajo impacto del cambio e incremento de la reutilización?	Asignar una responsabilidad de manera que el acoplamiento permanezca bajo	No afectan los cambios de otros componentes, es fácil entender la responsabilidad aisladamente, ayuda a reutilizar

Pago

Registro

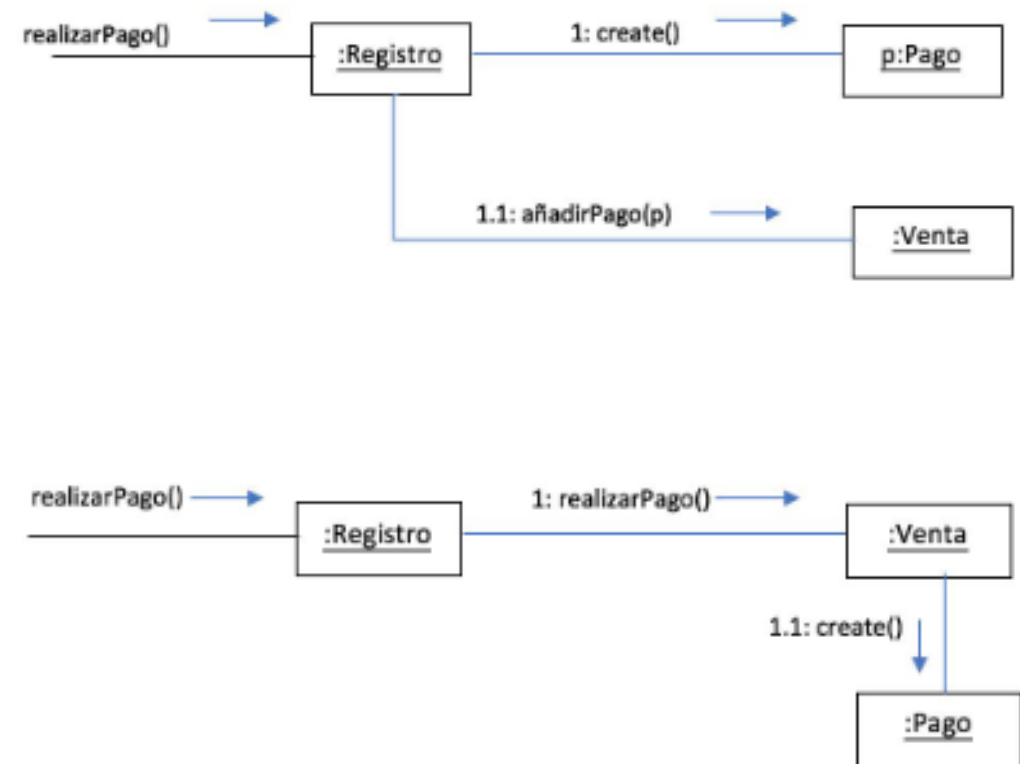
Venta

Si tenemos las clases: “Pago” , “Registro” y “Venta” y necesitamos crear una instancia de Pago y asociarla con la Venta. ¿Qué clase debería ser la responsable?

GRASP

BAJO ACOPLAMIENTO

Problema	Solución	Beneficios
¿Cómo soportar baja dependencias, bajo impacto del cambio e incremento de la reutilización?	Asignar una responsabilidad de manera que el acoplamiento permanezca bajo	No afectan los cambios de otros componentes, es fácil entender la responsabilidad aisladamente, ayuda a reutilizar



GRASP

BAJO ACOPLAMIENTO

- Formas comunes de acoplamiento en lenguajes orientados a objetos entre TipoX y TipoY:
 - TipoX tiene un atributo que referencia a instancia de TipoY o al propio TipoY
 - Un objeto TipoX invoca un objeto TipoY
 - TipoX tiene un método que referencia una instancia de TipoY de algún modo
 - TipoX es una subclase del TipoY
 - TipoY es una interfaz y TipoX la implementa

GRASP

ALTA COHESIÓN

Problema	Solución	Beneficios
¿Cómo mantener una complejidad manejable?	Asignar una responsabilidad de manera que la cohesión permanezca alta	Mayor claridad, mantenimiento simplificado, incrementa la reutilización

Pago

Registro

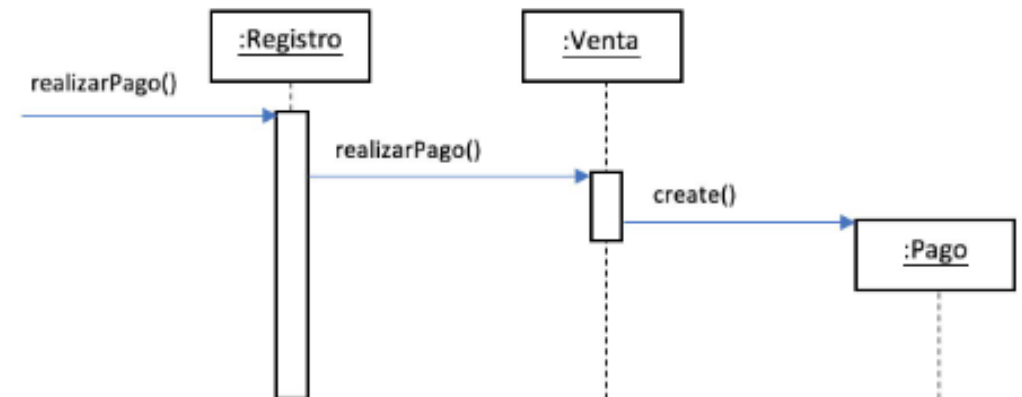
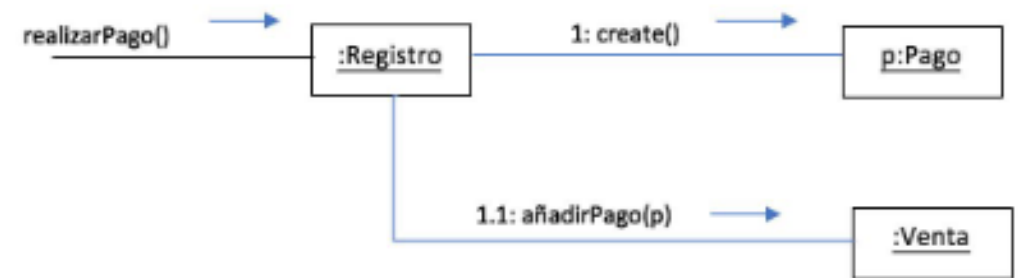
Venta

Si tenemos las clases: “Pago” , “Registro” y “Venta” y necesitamos crear una instancia de Pago y asociarla con la Venta. ¿Qué clase debería ser la responsable?

GRASP

ALTA COHESIÓN

Problema	Solución	Beneficios
¿Cómo mantener una complejidad manejable?	Asignar una responsabilidad de manera que la cohesión permanezca alta	Mayor claridad, mantenimiento simplificado, incrementa la reutilización



GRASP

ALTA COHESIÓN

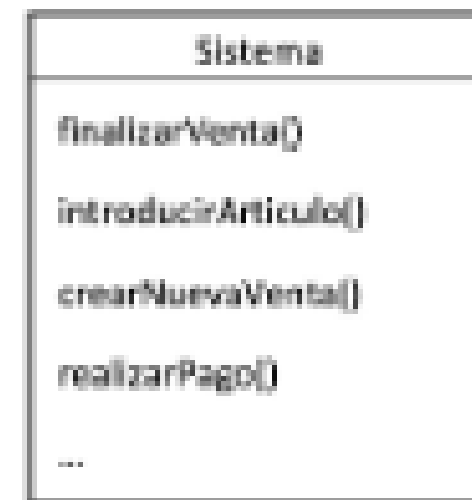
- Escenarios de grados de cohesión funcional:
 - Muy baja cohesión: una única clase es responsable de muchas cosas en áreas funcionales muy distintas
 - Baja cohesión: una única clase tiene la responsabilidad de una tarea compleja en un área funcional
 - Moderada cohesión: una clase tiene responsabilidades ligeras y únicas en pocas áreas funcionales distintas que están lógicamente relacionadas con el concepto de la clase, pero no entre ellas
 - Alta cohesión: una clase tiene una responsabilidad moderada en un área funcional y colabora con otras clases para llevar a cabo las tareas

Una clase con alta cohesión tiene un numero relativamente bajo de métodos con funcionalidad altamente relacionada y no realiza mucho trabajo

GRASP

CONTROLLER

Problema	Solución
¿Quién debe ser el responsable de gestionar un evento de entrada al sistema?	• Representa el sistema global, dispositivo o subsistema (controlador de fachada) • Representa un escenario de caso de uso en el que tiene lugar el evento del sistema, a menudo llamado <code><NombreCasoDeUso>Controller</code> (controlador de sesión) • Use la misma clase controlador de sesión para todos los eventos del sistema en el mismo escenario de caso de uso



Suponga las siguientes operaciones asociadas a eventos del sistema, ¿Quién debería ser el controlador de los eventos del sistema como `introducirArticulo` y `finalizarVenta`?

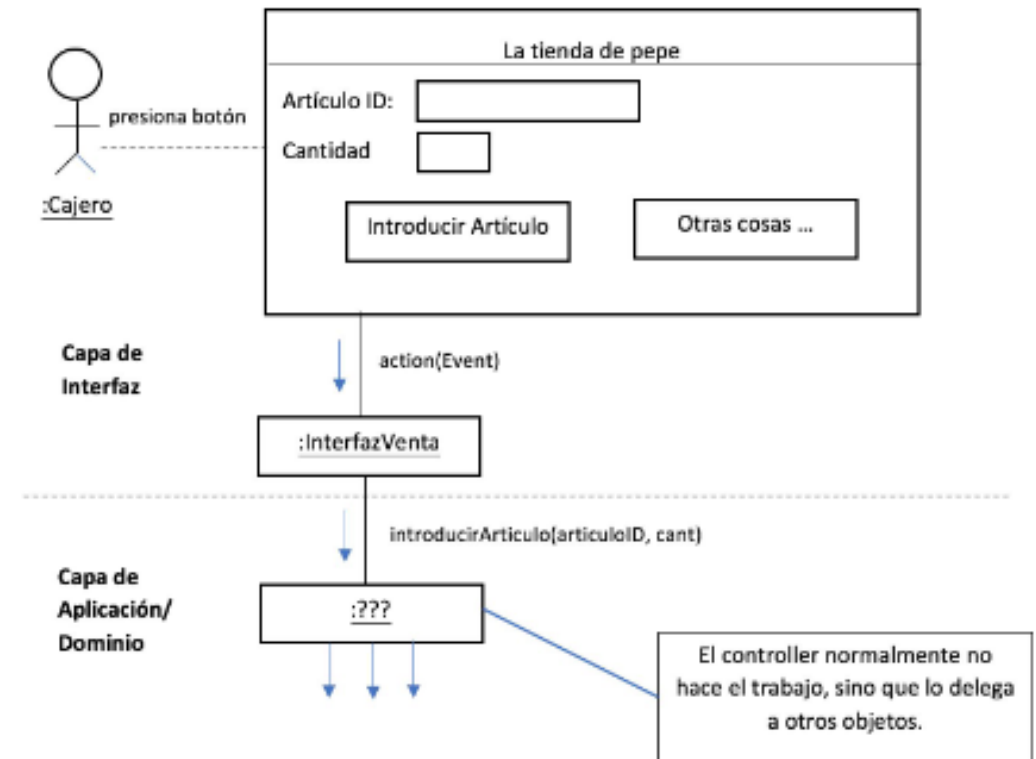
GRASP CONTROLLER

Problema

¿Quién debe ser el responsable de gestionar un evento de entrada al sistema?

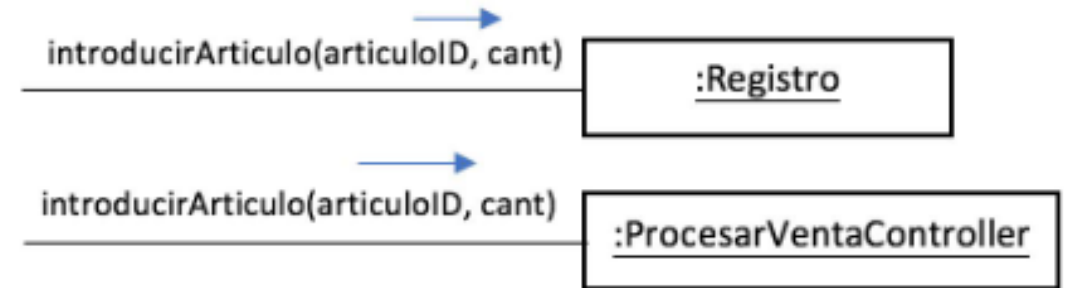
Solución

- Representa el sistema global, dispositivo o subsistema (controlador de fachada)
- Representa un escenario de caso de uso en el que tiene lugar el evento del sistema, a menudo llamado `<NombreCasoDeUso>Controller` (controlador de sesión)
 - Use la misma clase controlador de sesión para todos los eventos del sistema en el mismo escenario de caso de uso



GRASP CONTROLLER

Problema	Solución
¿Quién debe ser el responsable de gestionar un evento de entrada al sistema?	• Representa el sistema global, dispositivo o subsistema (controlador de fachada) • Representa un escenario de caso de uso en el que tiene lugar el evento del sistema, a menudo llamado <NombreCasoDeUso>Controller (controlador de sesión) • Use la misma clase controlador de sesión para todos los eventos del sistema en el mismo escenario de caso de uso



¿Cuál es el más apropiado?

GRASP CONTROLLER

Problema

¿Quién debe ser el responsable de gestionar un evento de entrada al sistema?

Solución

- Representa el sistema global, dispositivo o subsistema (controlador de fachada)
- Representa un escenario de caso de uso en el que tiene lugar el evento del sistema, a menudo llamado `<NombreCasoDeUso>Controller` (controlador de sesión)
 - Use la misma clase controlador de sesión para todos los eventos del sistema en el mismo escenario de caso de uso

ProcesarVentaController

```
finalizarVenta()  
introducirArticulo()  
crearNuevaVenta()  
realizarPago()
```

GestionarDevoluciones Controller

```
crearNuevaDevolucion()  
introducirArticuloDevuelto()
```

A veces conviene crear controladores que sean capaces de coordinar el trabajo relacionado con el caso de uso, en lugar de tener un único manejador gigante

BIBLIOGRAFÍA

- LARMAN, Craig; GARCÍA MOLINA, Jesús, UML y patrones: una introducción al análisis y diseño orientado a objetos y al proceso unificado. 2a ed. 2003. Madrid: Pearson Educación. xxv, 590 p. ISBN: 9788420534381. Capítulo 16.



ANÁLISIS Y DISEÑO ORIENTADO A OBJETOS

CLASE 3 - GRASP

